

Email Spam Detector

A local web application that checks pasted email content for common spam and phishing warning signs.

What this project does

Accepts an optional sender address and subject plus email text, checks the input in a Flask backend, and displays a rule-based classification, risk score, and detected signals in the browser.

1. Project overview

The Email Spam Detector is a small, locally run web application. A user pastes an email into the page; the browser sends the sender, subject, and body to a Flask endpoint. The backend scores matching warning signs and responds with a spam classification and explanation. The page checks again automatically after the user edits the email, and also provides a manual check button.

Important: This project currently uses regular-expression rules and weighted warning signals. It is not a trained machine-learning model, does not connect to an inbox, and cannot guarantee that a message is safe or malicious.

Features

- Optional sender email and subject fields, plus a required email-content field.
- Automatic analysis after a short typing pause, a manual Detect Spam button, and a Clear button.
- Spam/not-spam result, risk score from 0 to 100, and an explanation list of matched signals.
- Input validation and visible messages when a request fails or the backend is unavailable.
- Local-only operation: messages are analyzed by the local backend and are not sent to an email provider.

2. Application flow



The frontend is in `frontend/`. The Flask application and its tests are in `frontend/backend/`. The backend serves the frontend at `http://127.0.0.1:5000`, so the recommended flow uses one local origin.

3. Run the application

Requirements

- Python installed and available as the `python` command.
- Packages in `frontend/backend/requirements.txt` installed. Flask and Flask-Cors are used by the current app; scikit-learn, pandas, and joblib are listed but the current rule-based detector does not use them.

Windows quick start

1. Open the project folder in File Explorer.
2. Double-click `frontend/start_detector.bat`.
3. If Flask is not installed, open PowerShell in `frontend/backend` and run the install commands below.

4. The launcher starts the server and opens the detector. Keep its console window open while using the app.

Manual setup

```
cd frontend\backend
python -m pip install -r requirements.txt
python app.py
```

Then open <http://127.0.0.1:5000> . To stop the server, focus its console and press **Ctrl+C** .

Check that the backend is running

Open <http://127.0.0.1:5000/api/health> . A working server returns:

```
{"status": "ok"}
```

4. Use the detector

1. Optionally enter the sender email address and subject.
2. Paste the email text into **Email content**. The content field is required; the UI allows up to 10,000 characters.
3. Wait briefly for the automatic check, or select **Detect Spam**.
4. Read the classification, risk score, and signals. Select **Clear** to remove the fields and result.

Example input	Expected outcome	Why
"Congratulations! You are a winner and have won a free gift. Click here to claim your prize: https://bit.ly/example "	Spam likely	Several prize, click-request, link, and shortened-link signals exceed the threshold.
"Hi Alex, the meeting is scheduled for 10 AM tomorrow. Please bring the project notes."	No spam signals found	No configured warning pattern matches.

These are illustrative rule-engine examples, not guaranteed outcomes for every message.

5. Detection logic

The backend searches the combined sender, subject, and body text. Each listed rule adds its weight once if its pattern matches. A URL and a shortened URL can each add points; repeated exclamation marks add a small additional score.

Signal	Points	Examples / description
Prize, lottery, or free-gift language	20	winner, lottery, prize, free cash/gift/money/iPhone
Urgent or time-limited language	15	urgent, act now, immediately, limited time, expires today
Click or account verification request	25	click here, verify account/identity, confirm account/details
Sensitive credential or financial request	25	password, bank details, credit card, social security, OTP
Payment or money-transfer request	25	wire transfer, gift card, send money, payment required, processing fee
Guaranteed or risk-free claim	15	guaranteed, risk-free
Web link	10	http(s):// or www.
Shortened web link	15	bit.ly, tinyurl.com, t.co, goo.gl, cutt.ly
Repeated exclamation marks	5	Two or more consecutive ! marks

The raw score is capped at 100. A score of **30 or higher** is classified as `spam`; lower scores are classified as `not_spam`. A low score means the listed rules did not find strong signals; it is not proof of legitimacy.

6. Backend API

`POST /api/detect`

Accepts a JSON object. All three fields must be strings; sender and subject may be empty, but body must contain non-whitespace text.

```
{
  "sender": "offers@example.com",
  "subject": "Urgent prize",
  "body": "Congratulations winner! Click here to claim your prize: https://bit.ly/example"
}
```

Successful response:

```
{
  "is_spam": true,
  "classification": "spam",
  "risk_score": 85,
  "signals": [
    "Prize, lottery, or free-gift language",
    "Urgent or time-limited language",
    "A request to click a link or verify account details",
    "Contains a web link",
    "Contains a shortened link that hides its destination"
  ],
  "summary": "This email has multiple common spam or phishing warning signs."
}
```

`is_spam` is a boolean; `classification` is `spam` or `not_spam`; `risk_score` is an integer from 0 to 100; `signals` is a list of explanations; and `summary` is a readable result summary.

HTTP status	Meaning
200	Email successfully evaluated.
400	Invalid JSON object/field, invalid sender format, field length exceeded, or empty body.
415	Request is not JSON.
413	Request exceeds the server's 64 KB request limit.

`GET /api/health`

Returns `{"status": "ok"}` when the server is available.

7. Validation and troubleshooting

Run the backend tests

```
cd frontend\backend
python -m unittest
```

The test suite covers spam and ordinary messages, request validation, the health endpoint, frontend serving, and cross-origin requests from a local file page.

Symptom	Action
"Can't reach the spam checker" or no result	Start the server with <code>frontend/start_detector.bat</code> and open <code>http://127.0.0.1:5000</code> . Keep the launcher window open.
Flask is not installed	From <code>frontend\backend</code> , run <code>python -m pip install -r requirements.txt</code> .
Sender validation error	Enter a basic address such as <code>person@example.com</code> , or leave the optional sender field blank.
Empty-content error	Paste non-empty email text into the Email content field.
Unexpected classification	Review matched signals and remember that this prototype only checks its configured patterns; use an independent trusted channel to verify suspicious requests.

8. Privacy, security, and limitations

- The app does not log into a mailbox, retrieve messages, or forward pasted text to an external spam service.
- The development server binds to `127.0.0.1`, making it available on the local machine by default. It is not configured as a public production service.
- The API validates request types and lengths, validates a non-empty sender address, and limits the request body size.
- Rule matching is intentionally simple. It can miss novel phishing messages, and legitimate email may contain words or links that trigger signals.
- The risk score is a rule score, not a statistically calibrated probability of spam.
- The interface analyzes pasted text only. It does not inspect attachments, authenticate senders, resolve link destinations, or check domain reputation.

Safe-use guidance: Do not click an unexpected link or share passwords, one-time codes, or payment details because a message looks ordinary or receives a low score. Verify sensitive requests through a known, separate contact method.

9. Project files

Path	Purpose
<code>frontend/index.html</code>	Page layout and detector input/result elements.
<code>frontend/style.css</code>	Visual styling and responsive layout.
<code>frontend/script.js</code>	Automatic/manual checks, API calls, status and result rendering.
<code>frontend/start_detector.bat</code>	Windows quick-start launcher.
<code>frontend/backend/app.py</code>	Flask routes, request validation, and rule-based scoring.
<code>frontend/backend/requirements.txt</code>	Python package requirements.
<code>frontend/backend/test_app.py</code>	Backend API tests.
<code>frontend/backend/README.md</code>	Short backend setup and usage guide.